

AI Prompts & Workflow Documentation

Kohler Unified Enterprise AI Agent — Track 3 Case Study Submission

This document details every AI prompt, system instruction, and workflow decision used to design, build, debug, and deploy this project. The build used two AI tools in tandem: Claude for architecture planning and decision-making, and GitHub Copilot / Codex (agentic coding assistant in VS Code) for writing and executing the actual code, incrementally, one verified step at a time.

Overall Workflow Philosophy

The project was built using an incremental, verify-before-proceeding approach: each build step was scoped into a single focused prompt, sent to the coding agent, tested/validated, and only then was the next step requested. This kept the multi-agent RAG architecture debuggable at every stage rather than generating a large untested codebase at once.

1. Architecture Planning & Scoping (Claude)

Before any code was written, the full system architecture was planned and iteratively trimmed to a realistic solo-build scope, using Claude for design reasoning. Key planning decisions:

1.1 Original Architecture Diagram

The initial architecture (built in a diagramming tool) proposed the full enterprise vision: an Orchestration Layer (Multi-turn Context Resolver, Intent & Domain Classifier, Output Format Detector, Supervisor/Router), five Specialist Domain Agents (HR, Customer Support, Finance, Privacy, Legal/Compliance), a Domain Knowledge & Retrieval layer with per-domain vector indexes, a Governance & Guardrails layer (Grounding/Hallucination Check, PII Redaction, Policy & Compliance Filter), a Dynamic Output Formatting Engine (XML, JSON, Excel/CSV, Email, Markdown), Observability & Feedback (Human-in-the-loop Escalation, Audit Logging, Analytics Dashboard), and a Gateway & Identity layer (Auth/RBAC, Session Manager, multi-channel intake).

1.2 Scope-Trimming Decision Process

Claude was used to evaluate which architecture blocks added genuine build complexity versus which were safe to simplify, resulting in the following locked-in build scope:

KEPT (built for real):

- Multi-turn Context Resolver (via Streamlit session state)
- Intent & Domain Classifier + Supervisor/Router
- Specialist Agents + retrieval indexes (started at 3, expanded to 5)

- Foundation LLM (Gemini API)
- Output Format Detector + Formatters
- 1 live guardrail: Grounding/Hallucination Check
- Lightweight .jsonl audit logging

CUT (diagram-only / production roadmap, not built):

- Auth/RBAC (AD/SSO)
- Separate Session & Conversation State Manager service
- MS Teams/Slack/Email Intake channels
- Real Ingestion & ETL from SharePoint/Confluence/ServiceNow/Salesforce
- Human-in-the-Loop Escalation workflow
- Full Analytics Dashboard
- PII Redaction / Policy & Compliance Filter as separate pipelines (same grounding-check pattern noted as applicable, not implemented)

1.3 Tech Stack Decision

Final stack, decided after evaluating free-tier availability and solo-build complexity: Python, Google Gemini API (free tier LLM), ChromaDB (local vector database, no cloud account needed), sentence-transformers (local embeddings, no API dependency), and Streamlit (chat UI). Arize Phoenix (LLM observability) and LangChain were evaluated and explicitly excluded from the build to reduce dependency/integration risk under a tight solo timeline; a simple .jsonl log file was used instead for observability.

2. Incremental Build Prompts (Codex)

Each step below was sent as a distinct, scoped prompt to the coding agent (GitHub Copilot / Codex, in Agent mode inside VS Code), and confirmed working before the next was issued.

Step 1 — Environment Setup

Prompt sent to Codex:

I'm building a solo case-study project: a Kohler Unified Enterprise AI Agent —
a multi-domain conversational chatbot with an orchestrator agent, RAG specialist agents, and an output formatter agent.

TECH STACK:

- Python, plain (NO LangChain)
- Google Gemini API (free tier) for all LLM calls
- ChromaDB (local, no cloud account) as the vector database
- sentence-transformers for embeddings
- Streamlit for the chat UI

STEPS-

STEP 1: Environment setup — requirements.txt (google-generativeai, chromadb, sentence-transformers, streamlit) and a config file loading the Gemini API key from an environment variable (never hardcoded).

Result: config.py created with get_gemini_api_key(), requirements.txt locked. Verified with a manual test script confirming the key loads from the environment.

Step 2 — Document Loader + Chunker

Prompt sent to Codex:

Step 1 confirmed working (API key loads correctly). Move to Step 2: document loader + chunker for the three domains (HR Policy, Customer Support, Privacy Policy).

Result: document_loader.py — loads .txt files per domain, splits into ~250-word chunks with 50-word overlap, preserves domain/source/chunk_index metadata.

Step 3 — Embeddings + ChromaDB Storage

Prompt sent to Codex:

Fixed the environment — switched to Python 3.11 in a virtual environment, all dependencies (sentence-transformers, chromadb, etc.) installed successfully. Please retry Step 3: embed the chunks using sentence-transformers and store them in three separate ChromaDB collections, one per domain.

Result: vector_store.py — embeds each domain's chunks and stores in its own persistent Chroma collection. Note: an earlier attempt failed due to a Python 3.14 environment being

incompatible with sentence-transformers/torch; resolved by creating a dedicated Python 3.11 virtual environment.

Step 4 — Retrieval Function

Prompt sent to Codex:

Step 3 confirmed working — embeddings, ChromaDB storage, and retrieval all tested successfully. Move to Step 4: formalize the retrieval function so it takes any (domain, query) pair and returns the top 3-4 most relevant chunks with their similarity scores. Make sure it's a clean reusable function I can call from the orchestrator later.

Result: retriever.py — reusable retrieve(domain, query) function, tested against all three domains with live queries confirming correct top-match retrieval and source attribution.

Step 5 — Domain Classifier

Prompt sent to Codex:

Step 4 confirmed working across all three domains. Move to Step 5: build the domain classifier — a function that takes the user's current message plus recent conversation history, and uses Gemini to classify which domain it belongs to (HR, Customer Support, or Privacy Policy). It should detect when the domain changes mid-conversation, not just classify each message in isolation.

Result: domain_classifier.py — history-aware classifier using Gemini, validated against a topic-switch test (HR to Privacy mid-conversation) with correct re-routing.

Step 6 — Orchestrator/Router + Governance

Prompt sent to Codex:

Step 5 confirmed working — domain classifier handles topic switches, conversation history, and JSON output correctly. Move to Step 6: build the orchestrator/router. It should take the classifier's domain output, call that domain's retrieval function to get relevant chunks, then generate the final answer using Gemini with the retrieved chunks as context. Also bake in a strict governance instruction in the system prompt: never expose or repeat

sensitive customer personal data (names, account numbers, contact info, etc.) even if present in retrieved context — refuse and redirect if asked to reveal such data directly.

Result: orchestrator.py — combines classification, retrieval, generation, and a governance system-prompt instruction. Tested with a direct sensitive-data probe and confirmed it refuses/redirects rather than exposing information.

Step 7 — Output Formatter

Prompt sent to Codex:

Step 6 confirmed working, including governance refusal behavior. Move to Step 9: build the Output Format Detector — a function that reads the user's message and detects if they want "json", "email", or default "markdown" format. Then build the three formatters: markdown returns the answer as-is, email wraps it as a short professional email with subject line, and json returns {"answer": ..., "source_domain": ..., "confidence": ...} with validation that it parses as real JSON.

Result: output_formatter.py — format detector + 3 formatters, JSON round-trip parsing validated.

Step 8 — Live Faithfulness Check + Logging

Prompt sent to Codex:

Step 9 confirmed working. Move to Step 11: build a live grounding/fidelity check that runs after each answer — a second Gemini call asking "is this answer fully supported by the given retrieved context? yes/no + brief reason." Log the result (pass/fail + reason) alongside each query.

Result: faithfulness.py — post-hoc Gemini-based groundedness check, results appended to a local. jsonl log alongside each query/response/domain/format.

Step 9 — Streamlit UI

Prompt sent to Codex:

Step 11 confirmed working. Move to Step 13: build a simple Streamlit chat UI — single page, message history displayed top to bottom using

st.session_state for multi-turn conversation, text input at the bottom.

Wire it up to call the orchestrator for each message and display the formatted response. No sidebar config, no extra tabs — keep it minimal.

Result: app.py — functional single-page Streamlit chat interface, deployed locally at localhost:8501.

3. Scope Expansion — Matching Full Brief Requirements

After the core 3-domain / 3-format prototype was verified working end-to-end, the build was expanded to match the case study's full stated requirements (5 knowledge domains, richer output formats).

3.1 Adding Financial Guidelines + Legal/Compliance Domains

Prompt sent to Codex:

Add two new domains to the system, following the exact same pattern as the existing HR/Support/Privacy domains: "Financial Guidelines" and "Legal/Compliance". I've added sample documents at data/financial_guidelines/ and data/legal_compliance/. Please: 1) chunk and embed these into two new ChromaDB collections, 2) update the domain classifier to recognize these two new domains alongside the existing three, 3) update the orchestrator/router to call these domains correctly, 4) keep the same governance instructions applied to these new domains too. Test retrieval for both new domains before confirming.

Result: All 5 domains integrated; classifier validated across a full 5-way topic-switch conversation test.

3.2 Adding Excel/CSV and XML Output Formats

Prompt sent to Codex:

Add two more output formats to the existing Output Format Detector and Formatter, alongside the existing json/email/markdown: 1) "excel" - generate a downloadable CSV file (not full xlsx, keep it simple) containing the answer, source domain, and confidence, using Python's built-in csv module,

and provide a way for the Streamlit app to offer it as a download button, 2)

"xml" - format the answer as a simple XML structure with <response>

<answer>, <domain>, <confidence> tags. Update the Output Format Detector's

classification logic to recognize when a user asks for "excel", "csv",

"xml", or "download" and route to the correct formatter. Test all 5 formats

(markdown, email, json, excel, xml) with a sample query before confirming.

Result: 5 total output formats implemented and tested; CSV delivered via a Streamlit download button rather than inline text.

3.3 Model Compatibility Fixes

During development, the Gemini API deprecated several model versions mid-build (gemini-2.0-flash, then gemini-2.5-flash were both retired for new users). Rather than repeatedly hardcoding a guessed model name, the following prompt was used to make the fix self-correcting:

Prompt sent to Codex:

Instead of guessing a model name, please write a small script that calls `client.models.list()` to fetch the actual list of models my API key has access to that support `generateContent`, print them, and then update all Gemini calls in the codebase to use the first suitable flash model from that real list (not a hardcoded guess).

[Follow-up after another deprecation]:

Use "gemini-3.6-flash" as the model name everywhere in the codebase — it's confirmed available in the discovered models list and is what the API error explicitly recommends. Also add a fallback: if this model ever returns a 404 in the future, automatically retry with "gemini-3.5-flash" and then "gemini-2.5-flash-lite" before failing, so the app doesn't break again from a model deprecation.

Result: discover_gemini_models.py added; model selection made resilient to future API deprecations via a fallback chain.

3.4 UI Styling — Kohler Brand Colors

Kohler's official website was reviewed to identify its actual visual identity: a clean, minimal look built around white, black, a deep red/maroon accent, and warm neutral tones (used across Kohler's product finish photography, such as their "Dune" neutral tone). These colors were used as the basis for the app's UI theme, so the interface reflects the brand it was built for rather than a generic default look.

Prompt sent to Codex:

Restyle the Streamlit UI to match Kohler's actual brand identity, without changing any backend logic (orchestrator, RAG, formatting must stay untouched). Use this color palette, based on Kohler's real website:

- Primary background: white
- Primary text/headers: black
- Accent color (buttons, highlights, badges): deep red/maroon
- Secondary neutral tone (chat bubbles): warm beige/taupe
- Borders/dividers: light grey

Style the header/banner, subtitle, chat bubbles (user vs. AI visually distinct), domain badge pills next to each AI response, buttons, and a "Thinking..." loading spinner using this palette. Keep it clean and minimal, in line with a premium home-fixtures brand rather than a casual app.

Result: Custom CSS applied via st.markdown, using Kohler's actual brand colors; domain badges added as a visible proof of routing correctness during demos.

3.5 Deployment Fix (Streamlit Community Cloud)

Prompt sent to Codex:

The deployed app on Streamlit Cloud is failing with "Collection [customer_support] does not exist" because chroma_db/ isn't in the GitHub repo (it's gitignored). Please update app.py so that on startup, it checks if the ChromaDB collections exist; if they don't, automatically run the

same embedding/chunking pipeline (from `build_vector_store.py`) to build them from the `data/` folder before the app starts accepting questions.

Result: App auto-builds its vector database on first run in any fresh environment (local or cloud), removing the need to commit a pre-built binary database to version control.

4. Governance & Guardrail Design

As specified in the original architecture, governance was implemented via strict system-prompt instructions baked directly into the orchestrator and each specialist agent, rather than a separate filtering tool/service (per the project's stated MVP scope decision). The instruction pattern used:

"Never expose or repeat sensitive customer personal data (names, account numbers, contact info, etc.) even if present in retrieved context; refuse and redirect if asked to reveal such data directly."

This was tested by directly probing the deployed agent with requests for sensitive data (e.g. "what is the customer's account number") and confirming refusal/redirection rather than data exposure.

5. Observability Approach

RAGAS-style evaluation was scoped as: a live, lightweight per-query faithfulness check (built, see Step 8) prioritized over an offline batch-evaluation script (RAGAS proper, requiring a curated test set) due to solo-build time constraints — this trade-off decision was made explicitly rather than attempting both simultaneously. The live check runs a second Gemini call per response asking whether the answer is fully supported by retrieved context, logging pass/fail plus a brief reason to a local audit trail (.jsonl).

Summary

This project was built through roughly 15 distinct, sequential AI-assisted build steps, each scoped, executed by the coding agent, and independently verified before proceeding — combined with continuous architecture and troubleshooting guidance throughout. The final system implements all 5 required knowledge domains and 5 output formats from the original brief, with a working live faithfulness guardrail, prompt-based governance, and a deployed, publicly accessible chat interface.